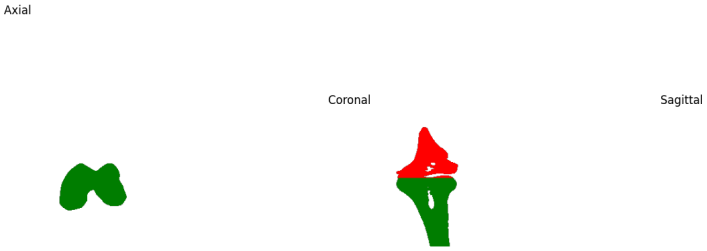
Task 3 Documentation (Sajjan Acharya)

Image Segmentation

For the purpose of segmentation, a 3D scan of the knee was provided. Use of Hounsfield scale, thresholding was used with filters like gaussian filter and bilateral filter to capture necessary edges. In order to fill around the cavities in the bone, propagation technique was used.
For separation of two bones, use of labels was used, and the size of the regions were used to separate the bones. After separation, different colors were used to separate femur and tibia.



Inflation of 2D model to 3D

From torchvision.models, Densenet121 was obtained with pretrained weights. The objective was to inflate all 2D convolution layers into 3D layers, in the hope that it can analyze our 3D regions for femur, tibia and just the background. However, it was found out that the normalization layers, and the pooling layers will have to be inflated to 3D too, to handle the 3D input tensors.
Also, for the output of the densenet, adaptive average pooling was again converted to 3D, since it didn't fall under the module children. Thus, this was how the Densenet was modified to handle 3D tensors.

Tensor Downsample:

The tensor shape for regions were all (216, 512, 512). And to make the tensors valid as input to Densenet, there had to be 3 channels. Thus, the shape would have to be (1,3,216,512,512). But when I tried to infer this shape, my memory ran out. It required more than 31 Gigs of RAM. Hence, I downsampled all 3 regions to (216, 216, 216) with interpolation. And with permutation and reshaping, the regions were modified to the shape of (1, 3, 216, 216, 216). This was the shape of all 3 regions while inferring the 3D-converted model.

Required Convolution Layers:

As per the question, we were required to get the feature maps from the output of "last", "third-last" and "fifth last" convolution layers in DenseNet. The last 10 convolution layers from

the model were:

```
features.denseblock4.denselayer12.conv1
features.denseblock4.denselayer12.conv2
features.denseblock4.denselayer13.conv1
features.denseblock4.denselayer13.conv2
features.denseblock4.denselayer14.conv1
features.denseblock4.denselayer14.conv2
features.denseblock4.denselayer15.conv1
features.denseblock4.denselayer15.conv2
features.denseblock4.denselayer16.conv1
Features.denseblock4.denselayer16.conv2
```

From them, the required layers were:

```
features.denseblock4.denselayer14.conv2
features.denseblock4.denselayer15.conv2
Features.denseblock4.denselayer16.conv2
```

These layers were chosen, and outputs from them were stored to get the cosine similarities.
All the features, for each layer, were average pooled into a vector of dimension [32]. This dimension was standard for all 3 regions, such that the computation of cosine similarity would be consistent.

Cosine Similarities and Analysis

For 3 regions, tibia, femur and the background, one dimensional vector of shape 32 was obtained for each of the 3 selected convolution layers. And cosine similarity was computed between each of these regions, for all layers. The similarity score achieved is as follows:

Pair	Last Layer	Third-Last Layer	Fifth-Last Layer
Tibia-Femur	0.882862	0.974867	0.866306
Tibia-Background	0.205657	0.684762	0.160198
Femur-Background	0.328276	0.653026	0.020768

Furthermore, the **norms** were computed for the features extracted from each of those 3 layers.

```
tibia        last         norm = 0.4247
tibia        third_last   norm = 0.7982
tibia        fifth_last   norm = 0.2218
femur        last         norm = 0.3536
femur        third_last   norm = 1.1171
femur        fifth_last   norm = 0.2699
background   last         norm = 6.8644
background   third_last   norm = 6.7134
background   fifth_last   norm = 1.7444
```

Comparatively, tibia and femur have lower norms, as their spatial regions are smaller. In contrast, background has a higher norm, since it has a large spatial area.

For **mean and standard deviation** between features for all 3 regions, the values were obtained as follows:

Layer: last
tibia ↔ femur: mean=0.0290, std=0.0208, max=0.0841
tibia ↔ background: mean=0.8799, std=0.8294, max=3.0546
femur ↔ background: mean=0.8809, std=0.8195, max=3.0363

Layer: third_last
tibia ↔ femur: mean=0.0463, std=0.0501, max=0.2321
tibia ↔ background: mean=0.5788, std=0.9444, max=4.9769
femur ↔ background: mean=0.5718, std=0.9169, max=4.8705

Layer: fifth_last
tibia ↔ femur: mean=0.0187, std=0.0151, max=0.0683
tibia ↔ background: mean=0.2352, std=0.1966, max=0.8349
femur ↔ background: mean=0.2375, std=0.2041, max=0.8585

Femur and Tibia are almost numerically identical vectors, so less standard deviation, but bones with background have higher standard deviation.